NEW-AGE GLOBAL UNIVERSITY FOR LIBERAL EDUCATION

Semester: II
Operating System
Category: Core
Course Code: CS1103
(Theory and Lab)

Course Outline

Unit – I	04 Hrs
Introduction to Operating Systems: Operating Systems –Definition Types- Functions -Abstract view of OS- System Structures – System Calls- Virtual Machines.	
Unit – II	08 Hrs
Process Management: Process Concepts, Inter-process communication –Threads –Multithreading, Process Scheduling, First-Come, First-Served (FCFS), Shortest Job First, Round Robin (RR), Priority Scheduling.	
Unit – III	06 Hrs
Synchronization and Deadlocks: Synchronization –Semaphores –Monitors, Hardware Synchronization – Deadlocks –Methods for Handling Deadlocks.	
Unit – IV	12 Hrs
Memory Management: Memory Management Strategies –Contiguous and Non-Contiguous allocation –Paging –Virtual memory Management –File System Design and Implementation – Mass Storage Structure –Disk Scheduling and Management, FCFS, SSTF, and SCAN -I/O Systems- Overview of System Protection and Security.	

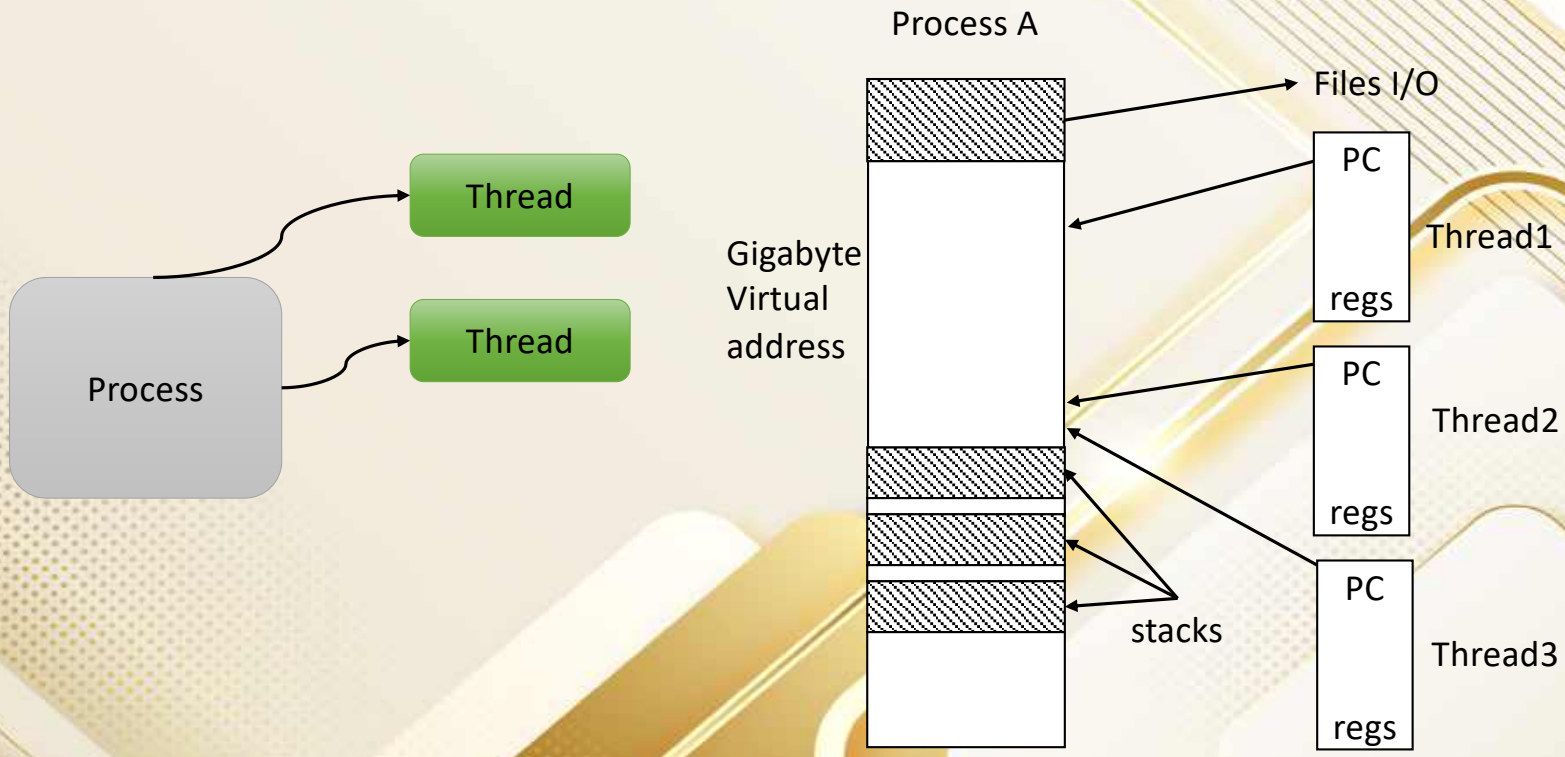
Contents

- Process and Thread
- Layout of a process in memory
- Few key concepts of memory
- What is a process?
- Process model
- Process versus thread
- Memory layout of a C program
- Activity: Memory segment display
- Process state: 5 state model
- Need for a suspend state
- Process scheduling:

NOTE: Session 2.2- Two and Three process state model

Process and Thread

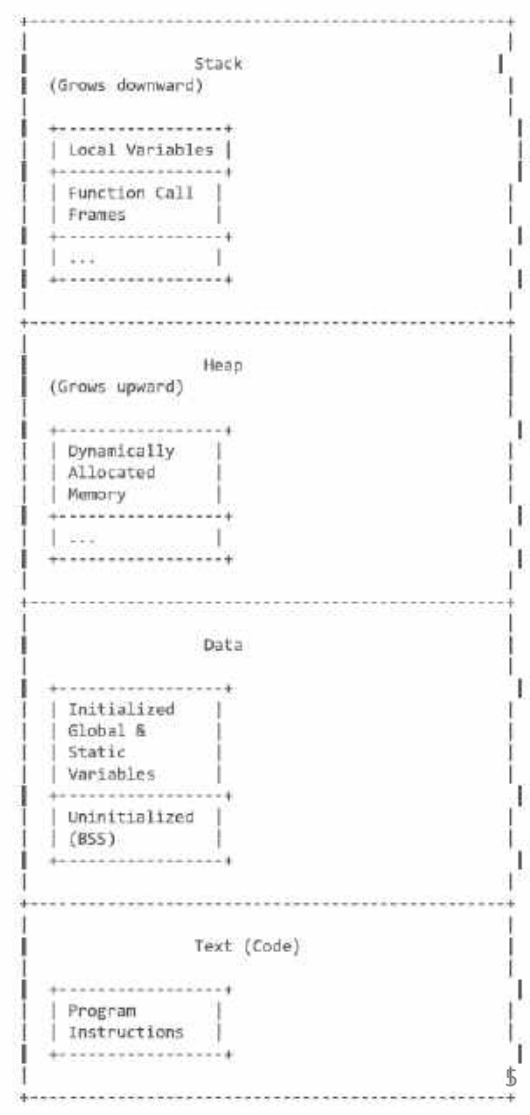
Process is a program which is executing some code and a thread is an independent path of execution in the process.



Layout of a process in memory

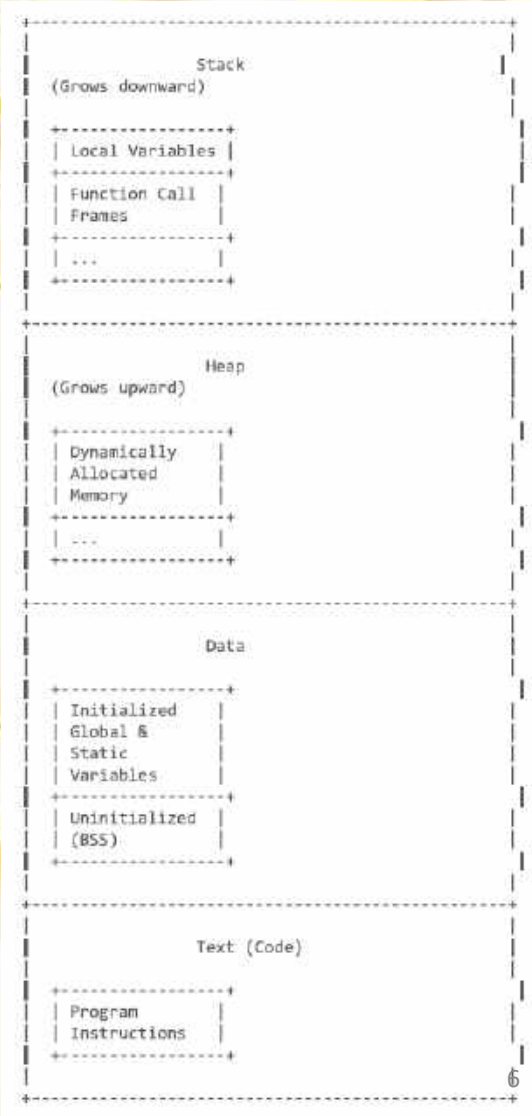
Explanation of Memory Layout:

- **Text (Code) Segment:**
  - Contains the program's executable instructions (machine code).
  - This segment is usually read-only to prevent accidental modification of the code.
  - It's often shared among multiple processes running the same program.
- **Data Segment:**
  - Stores global and static variables that are initialized before the program starts execution.
  - Also includes the BSS (Block Started by Symbol) section, which holds uninitialized global and static variables. These are typically initialized to zero by the system.



Layout of a process in memory

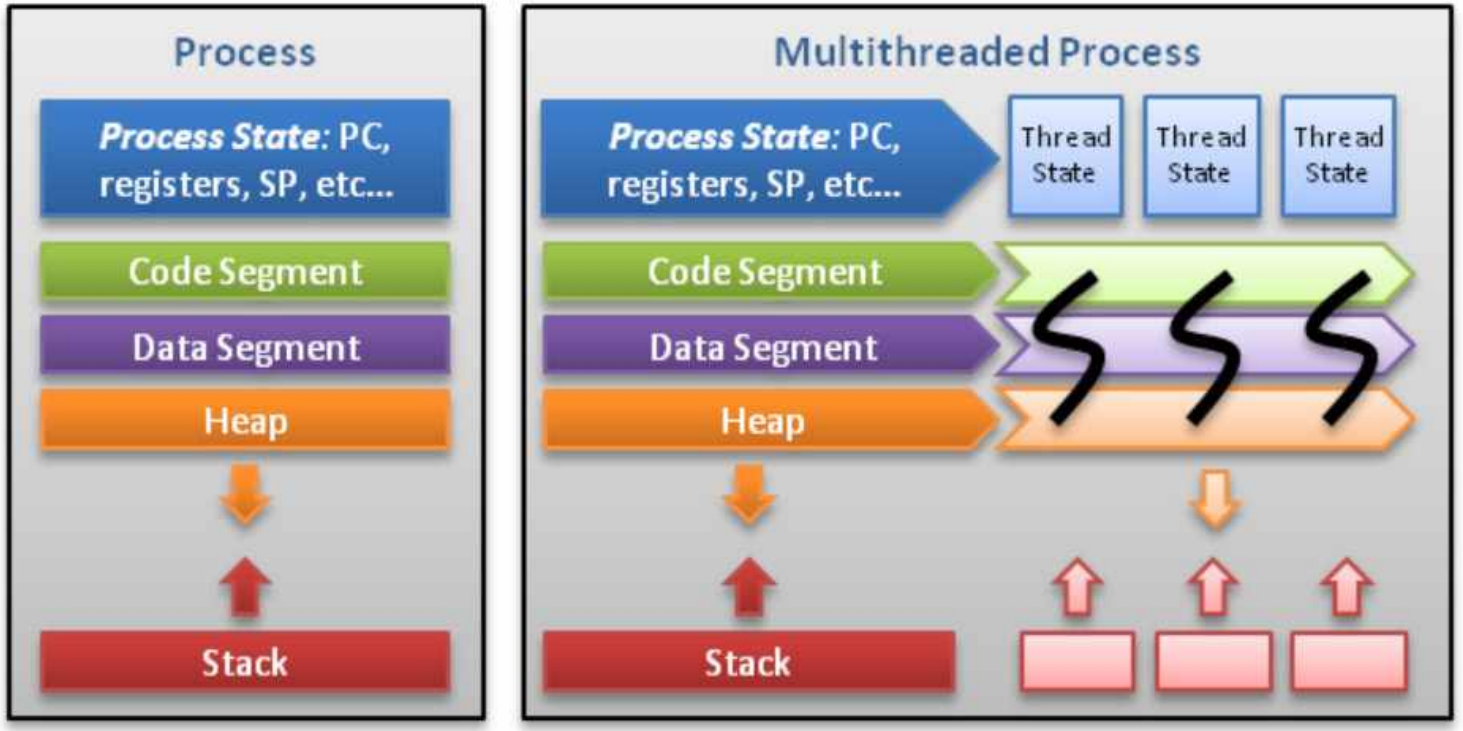
- **Heap Segment:**
  - Used for dynamic memory allocation during program execution.
  - Memory is allocated and deallocated using functions like malloc() and free() (in C/C++).
  - The heap grows upward in memory.
- **Stack Segment:**
  - Used for storing local variables, function call frames (including parameters and return addresses), and temporary values.
  - The stack grows downward in memory.
  - Each function call creates a new stack frame. When a function returns, its frame is popped off the stack.



Key concepts:

- **Virtual Address Space:** Each process has its own virtual address space, which is an illusion of having exclusive access to memory. The operating system translates these virtual addresses to physical addresses in RAM.
- **Memory Management:** The operating system is responsible for managing the allocation and deallocation of memory to processes.
- **Dynamic Memory Allocation:** The ability to allocate memory during program execution, as needed.
- **Stack Overflow:** Occurs when the stack grows beyond its allocated limit, often due to excessive recursion or too many local variables.
- **Memory Leak:** Occurs when dynamically allocated memory is no longer needed but is not freed, leading to a gradual depletion of available memory.
- This diagram represents a simplified view of process memory layout. The actual organization might be more complex depending on the operating system and architecture. For example, some systems have a separate segment for shared libraries. However, this provides a good general understanding.

Processes and threads

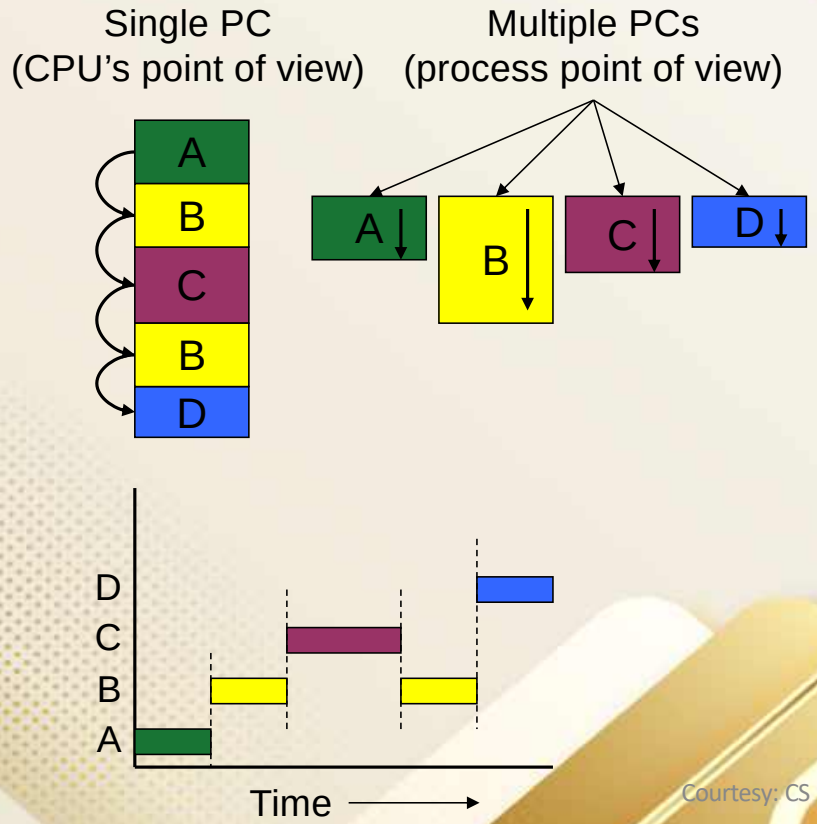


Threads contain only necessary information, such as a stack (for local variables, function arguments, return values), a copy of the registers, program counter and any thread-specific data to allow them to be scheduled individually. Other data is shared within the process between all threads.

What is a process?

- Code, data, and stack
  - Usually (but not always) has its own address space
- Program state
  - CPU registers
  - Program counter (current location in the code)
  - Stack pointer
- Only one process can be running in the CPU at any given time!

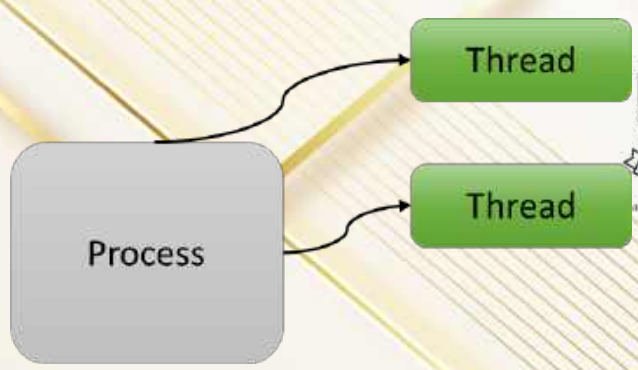
The process model



- Multiprogramming of four programs
- Conceptual model
  - 4 independent processes
  - Processes run sequentially
- Only one program active at any instant!
  - That instant can be very short…

Courtesy: CS 1550, cs.pitt.edu (originaly modified by Ethan L. Miller and Scott A. Brandt)

Processes and Threads



- A process is an executing instance of an application.
- Thus, the essential difference between a thread and a process is the work that each one is used to accomplish.
- Threads are used for small tasks, whereas processes are used for more 'heavyweight' tasks – basically the execution of application.
- Technically, most significant difference between processes and threads is with respect to their address space and context switching.
- All threads from a process share the same address space but processes have their own address space.
- Similarly, context switching between processes is more expensive than context switching between threads belonging to the same process.
- Expensive means it takes more time because the context to be saved and restored is more in process context switch.

Processes and Threads

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Thread is a Light-Weight Process with a reduced state.
- State reduction is achieved by having a group of related threads share other resources such as memory and files.
- In thread-based systems, threads take over the role of processes as a smallest individual unit of scheduling.
- In such a system, the process or task serves as the environment for execution of threads.
- The process thus becomes a unit of resource ownership, such as memory or file, for a collection of threads.
- A process with exactly one thread is equivalent to a classical process.

Created by: Dr. Sangeeta Bhanja Chaudhuri

Processes versus Threads

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

1. Processes are used when the tasks are essentially unrelated
2. Each process has its own address space and PCB
3. Address spaces are protected from each other
4. Switching between the process is done at the kernel level
5. Owned by one or more users

1. Threads are used when the tasks are actually part of the same job
2. Threads belonging to the same process share the process' address space, code data, and files , but not the registers and the stack.
3. No address space protections

   Note: Among the threads of the same process.
4. Switching between the threads can be done either at the user level or the kernel level.
5. Usually owned by a single user.

   Note: Threads belonging to a process

Processes and Threads

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Each thread belongs to exactly one process, and no thread can exist outside of a process.
- Processes are static, and only threads can be scheduled for execution, within an application.
- Typically, each thread represents a separate flow of control and is characterized by its own stack and HW state.
- Since all resources other than processor is managed by the enclosing process, switching between related threads is fast and efficient.
- However, switching between threads that belong to different processes incurs full process switch, along with its associated overhead (file/memory mgt).

Memory layout of a C program

Dynamic memory layout

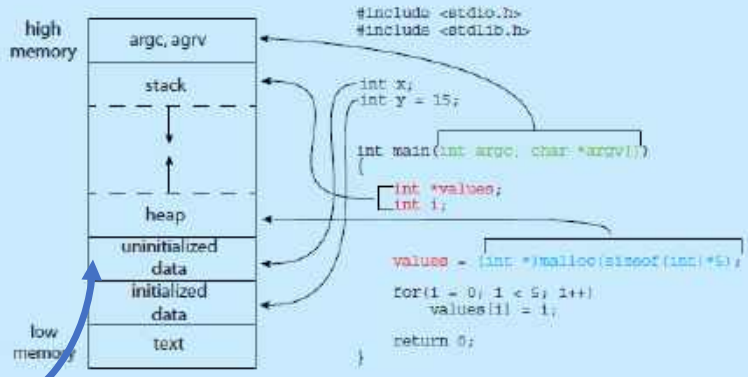
Static memory layout

Initialized to zero at run time

MEMORY LAYOUT OF A C PROGRAM

The figure shown below illustrates the layout of a C program in memory, highlighting how the different sections of a process relate to an actual C program. This figure is similar to the general concept of a process in memory as shown in Figure 3.1, with a few differences:

- The global data section is divided into different sections for (a) initialized data and (b) uninitialized data.
- A separate section is provided for the `argc` and `argv` parameters passed to the `main()` function.



The GNU `size` command can be used to determine the size (in bytes) of some of these sections. Assuming the name of the executable file of the above C program is `memory`, the following is the output generated by entering the command `size memory`:

```
text    data    bss     dec     hex     filename
1158    284     8       1450    5aa     memory
```

The `data` field refers to uninitialized data, and `bss` refers to initialized data. (`bss` is a historical term referring to *block started by symbol*.) The `dec` and `hex` values are the sum of the three sections represented in decimal and hexadecimal, respectively.

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Memory layout of a C program

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- The memory layout of C program is divided into several segments:
  - Text segment (stores compiled code): Stores the program's executable instructions and constants like string_lit.
  - Data segment (stores global and static variables): Holds initialized global and static variables such as init_global.
  - BSS segment (uninitialized global and static variables): Contains uninitialized global and static variables like global_v.
  - Heap (dynamically allocated memory): Manages dynamically allocated memory using malloc and free, as shown with heap_v.
  - Stack (stores local variables and function call information): Stores local variables within functions, for instance, stack_v in main.

This structure is crucial for efficient memory management

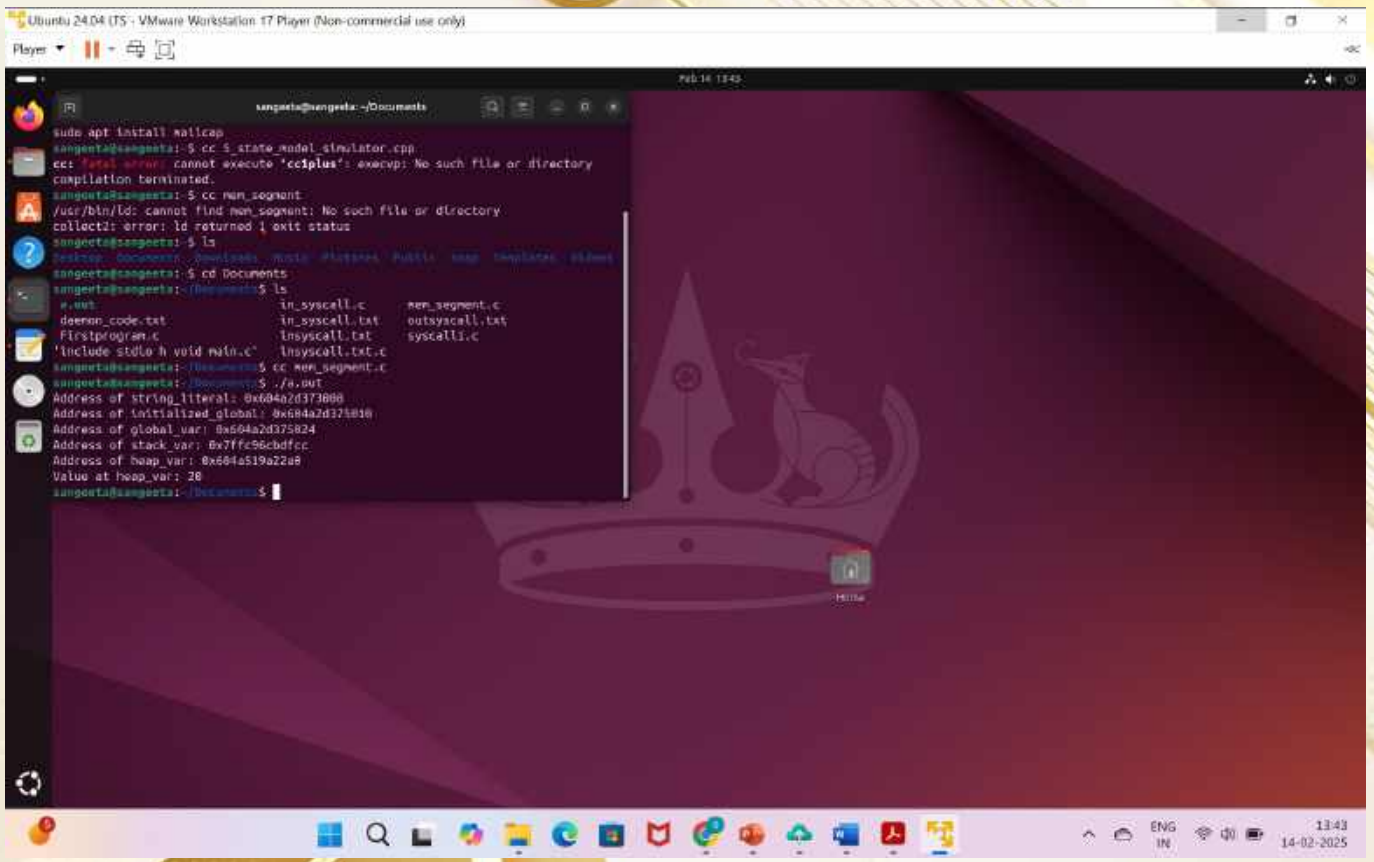
Activity: Memory segment display

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Filename: mem_segment.c

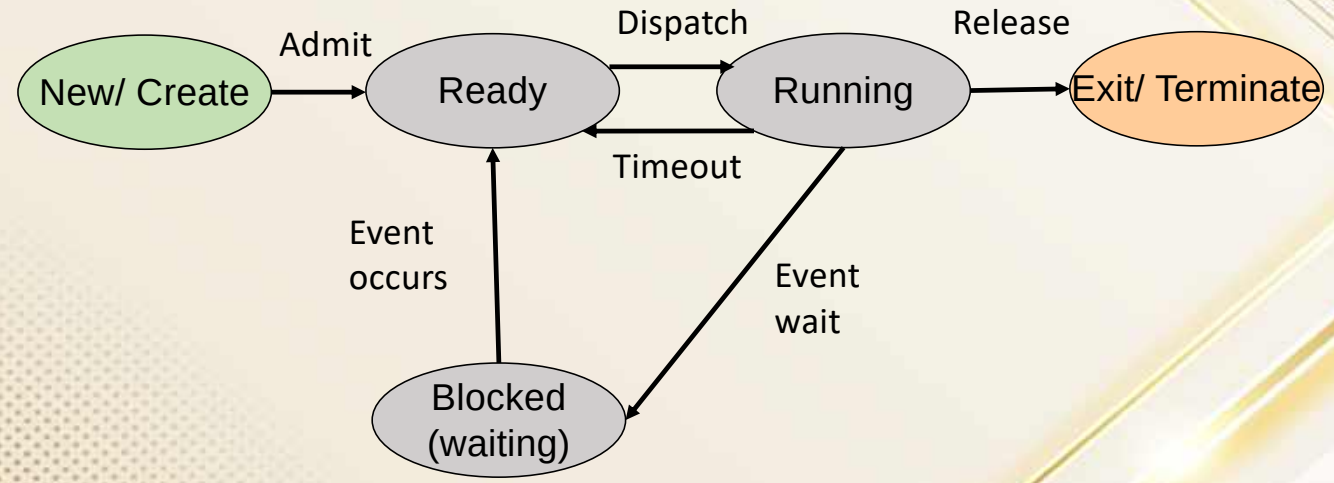
Compile: cc mem_segment.c

Output: ./a.out



Process state

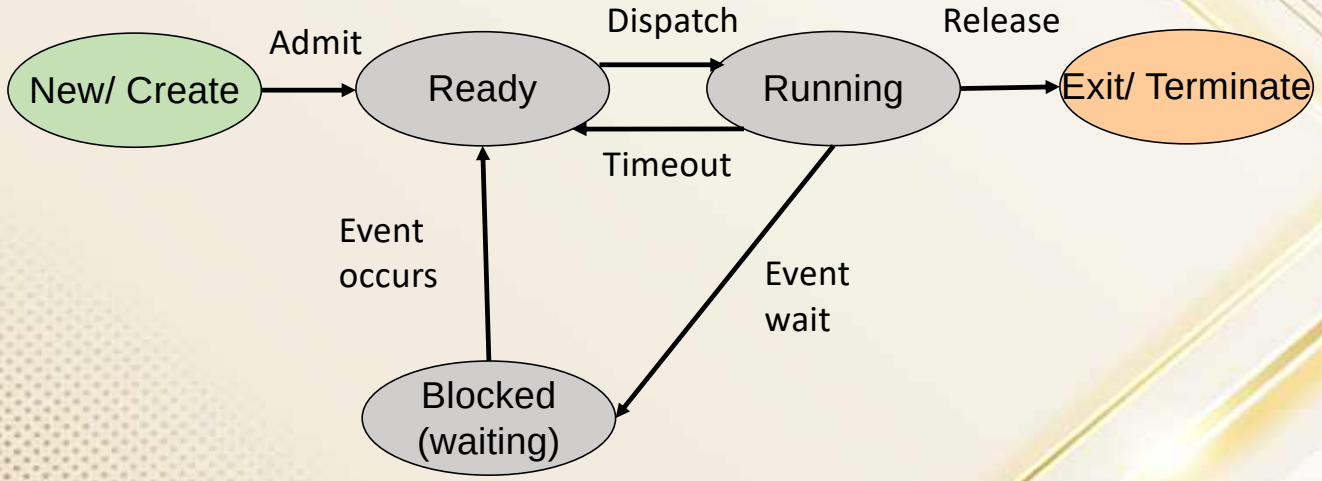
Five-state Model: Definition of States



- When a new process is created, a new PCB entry is created by the OS, but still the resident set of the program is yet to be loaded into the memory, thus, OS cannot run it, so, the process is kept in “New” state.
- When the loading of the process onto the memory is completed, the process moves from “New” to “Ready” state.

Process state

Five-state Model: Definition of States

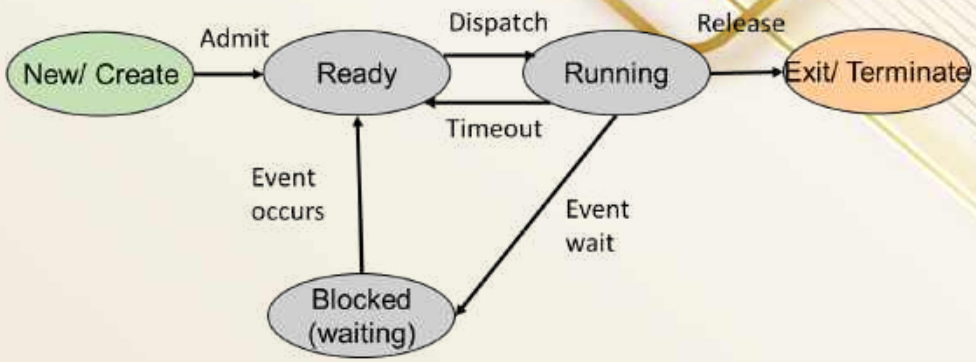


Running: The process that is currently being executed. Here, let us assume that there is a single processor, so at most one process at a time can be in this state.

Ready: A process that is prepared to execute when given the opportunity, that is given the CPU.

Blocked/Waiting: A process that cannot execute until some event occurs, such as the completion of an I/O operation.

Five-state Model: Definition of States

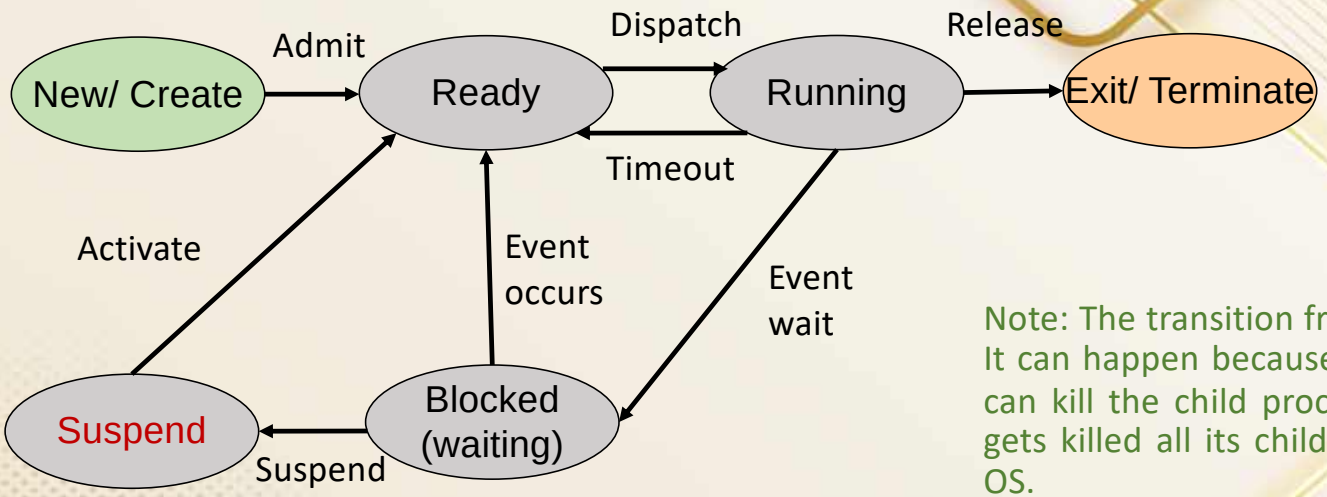


New: A process that has just been created but has not yet been admitted to the pool of executable processes by the OS. Typically, a new process has not yet been loaded into main memory, although its process control block (PCB) has been created.

Exit: A process could have been released from the pool of executable processes by the OS, either because it halted or because it aborted for some reason. The OS can extract any useful auditing information from the PCB of the exiting process, before releasing the PCB and removing the process fully from the system.

Note: The Exit status helps the process to be in the system, occupying the PCB and memory after it is terminated. This is helpful in case the same process is created by the user again, It's image is readily available on MM, so, state can be changed from Exit to New state saving time of loading it again from HDD.

Need for a **Suspend** state



Note: The transition from Any state ->Exit is not shown. It can happen because OS or any other parent process can kill the child process or when the parent process gets killed all its child processes also get killed by the OS.

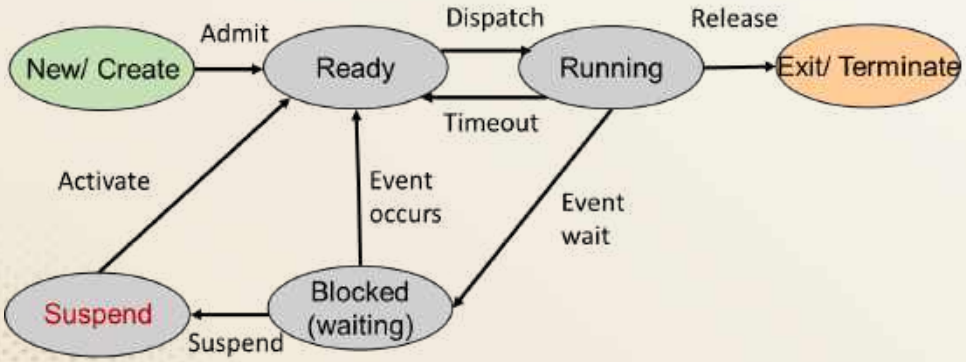
Imagine a situation where the main memory (MM) is filled with many processes, which are all in blocked state (waiting for an external event)

It implies that the CPU may run out of eligible Ready processes to keep it busy.

It could also prevent creation of new processes by either OS or by any of the currently running processes, because of lack of space in MM to keep them (i.e., keep the resident-sets of active processes).

To make the system efficient and responsive MM needs to be freed up. Let us understand how this additional state (Suspend) helps this purpose, next …

Suspend state Reasoning



Note: Priority of the process plays a role in the OS choosing one of the Blocked process to Suspend state. The lowest priority process is chosen for moving to Suspend state and then to disk.

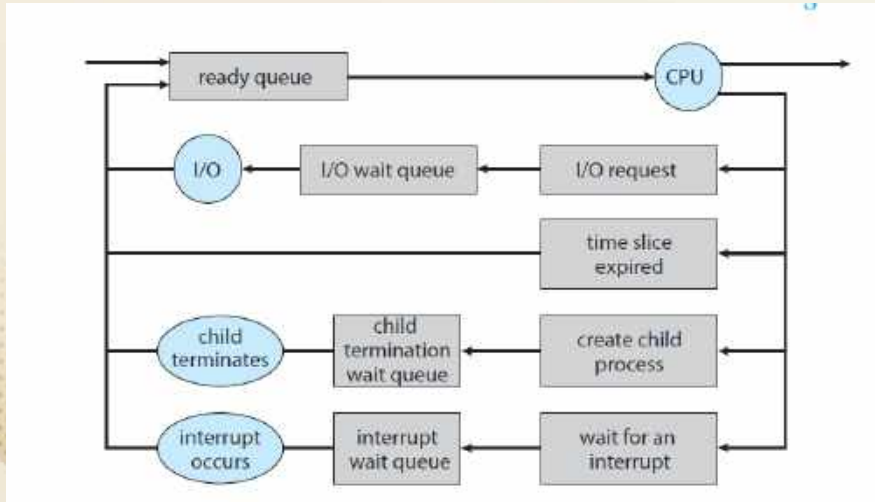
- OS chooses one of the Blocked processes residing in MM and change it to Suspend state, and moves the current process contents to disk.
- The space that is freed up in main memory can then be used to bring in some other processes or allow creation of new processes by the running processes.
- The suspended task is later brought into MM from the disk, directly into Ready state, when the event which it was blocked for, happens.

Process Scheduling

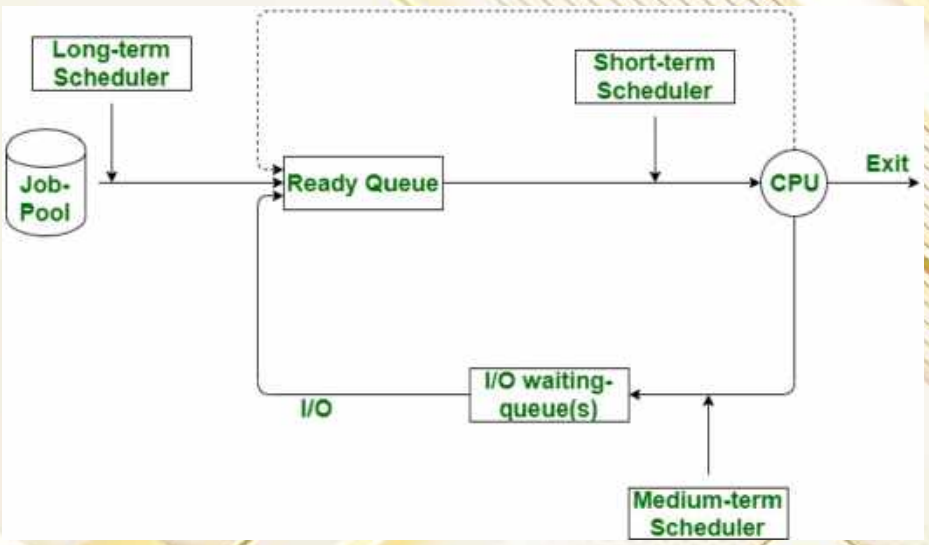
Things to Know:

- An I/O-bound process is one that spends more of its time doing I/O than it spends doing computations.
- A CPU-bound process, in contrast, generates I/O requests infrequently, using more of its time doing computations.

Scheduling Queues and schedulers



Queueing-diagram representation of process scheduling



Schedulers: Long, medium and short term

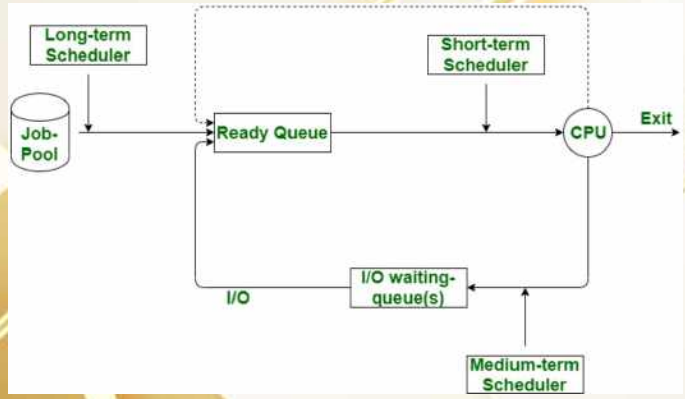
Schedulers

Short Term Scheduler(CPU Schedular):

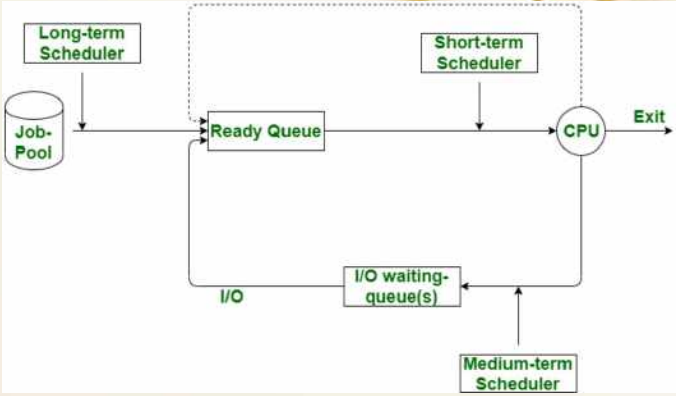
- Select which processes should be executed next and allocate to CPU
- The main objective is to increase system performance accordance with chosen set of criteria
- It is the change of ready state to running state of the process.

The short-term scheduler selects processes from the ready queue that are residing in the main memory and allocates CPU to one of them. Thus, it plans the scheduling of the processes that are in a ready state. It is also known as a CPU scheduler. As compared to long-term schedulers, a short-term scheduler has to be used very often i. e. the frequency of execution of short-term schedulers is high. The short-term scheduler is invoked whenever an event occurs. Such an event may lead to the interruption of the current process or it may provide an opportunity to preempt the currently running process in favor of another. The example of such events are:

- Clock ticks (time-based interrupts)
- I/O interrupts and I/O completions
- Operating system calls
- Sending and receiving of signals
- Activation of the interactive program



Medium Term schedulers



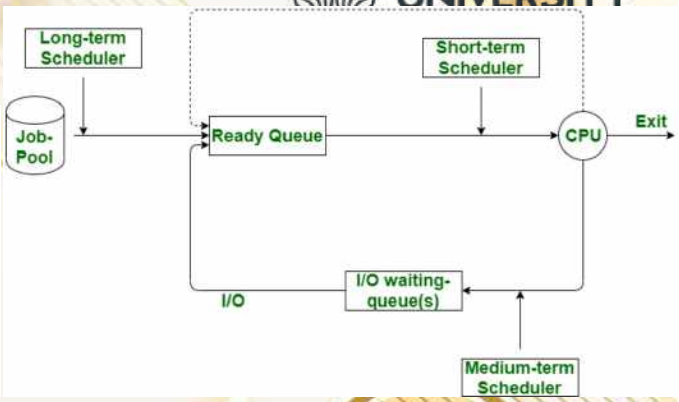
The medium-term scheduler is required at the times when a suspended or swapped-out process is to be brought into a pool of ready processes. A running process may be suspended because of an I/O request or by a system call. Such a suspended process is then removed from the main memory and is stored in a swapped queue in the secondary memory in order to create a space for some other process in the main memory. This is done because there is a limit on the number of active processes that can reside in the main memory. The medium-term scheduler is in charge of handling the swapped-out process. It has nothing to do with when a process remains suspended. However, once the suspending condition is removed, the medium terms scheduler attempts to allocate the required amount of main memory and swap the process in and make it ready. Thus, the medium-term scheduler plans the CPU scheduling for processes that have been waiting for the completion of another process or an I/O task.

Long Term schedulers

Schedulers:

Long Term Scheduler(Job Schedular):

- Select which processes should be brought into the ready queue
- The primary objective is to provide balanced mix of jobs such as I/O bound and processor bound.
- It also control the degree of multiprogramming



The long-term scheduler works with the batch queue and selects the next batch job to be executed. Thus, it plans the CPU scheduling for batch jobs. Processes, which are resource intensive and have a low priority are called batch jobs. These jobs are executed in a group or bunch. For example, a user requests for printing a bunch of files. We can also say that a long-term scheduler selects the processes or jobs from secondary storage device eg, a disk and loads them into the memory for execution. It is also known as a job scheduler. The long-term scheduler is called “long-term” because the time for which the scheduling is valid is long. This scheduler shows the best performance by selecting a good process mix of I/O-bound and CPU-bound processes. I/O bound processes are those that spend most of their time in I/O than computing. A CPU-bound process is one that spends most of its time in computations rather than generating I/O requests.

Referred materials

- Silberschatz, Galvin, Gagne, Operating System Concepts, John Wiley and Sons, 10th edition, 2023, ISBN: 935746056X
- Modern Operating Systems by Tanenbaum and Bos , 4th edition, Pearson Publisher, ISBN-13: 978-0-13-359162-0
- Courtesy: Prof. Mouli Sankaran, RV University.
- Courtesy: CS 1550, cs.pitt.edu (originally modified by Ethan L. Miller and Scott A. Brandt)